

qemu + gdb: The efficient way to understand/debug Linux kernel code/data structure

Adrian Huang | Aug, 2022

- * Based on kernel 5.11 (x86_64) – QEMU
- * 2-socket CPUs (4 cores/socket)
- * 16GB memory
- * Kernel parameter: nokaslr norandmaps
- * KASAN: disabled
- * Userspace: ASLR is disabled
- * Host OS: Ubuntu 20.04.1

Environment Preparation

- Repo: <https://github.com/AdrianHuang/gdb-linux-real-mode.git>

	AdrianHuang qemu: Enable 2-socket CPUs	ac1a148 on Jun 14	 16 commits
	conf	kernel config: Enable swap and NUMA	2 months ago
	gdb-files	gdb: Remove unused variables and print correct regi...	16 months ago
	scripts	qemu: Enable 2-socket CPUs	2 months ago
	README.md	Update README.md	2 years ago

Environment Preparation

- Repo: <https://github.com/AdrianHuang/gdb-linux-real-mode.git>
 - ./scripts/build.sh
 - Download/build kernel and busybox: Make sure if your machine can access the Internet
 - ./scripts/launch-vm.sh
 - Launch a guest OS (QEMU) and wait for gdb connection
 - ./scripts/launch-gdb.sh
 - Launch gdb debugger and connect to the QEMU guest OS
- Steps:

```
$ git clone https://github.com/AdrianHuang/gdb-linux-real-mode.git
Cloning into 'gdb-linux-real-mode'...
```

```
$ cd gdb-linux-real-mode/
```

```
$ ./scripts/build.sh
```

```
# After build.sh is done, you're all set!
```

QEMU/gdb at a glance: Console #1

Console #1: wait for gdb debugger

```
adrian@adrian-ubuntu:gdb-linux-real-mode$ ./scripts/launch-vm.sh
```



QEMU/gdb at a glance: Console #2: Real-mode entry point

Console #2: gdb debugger – breakpoint @0x10200 (real-mode entry point)

```
adrian@adrian-ubuntu:gdb-linux-real-mode$ ./scripts/launch-gdb.sh
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.1) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
```

↓

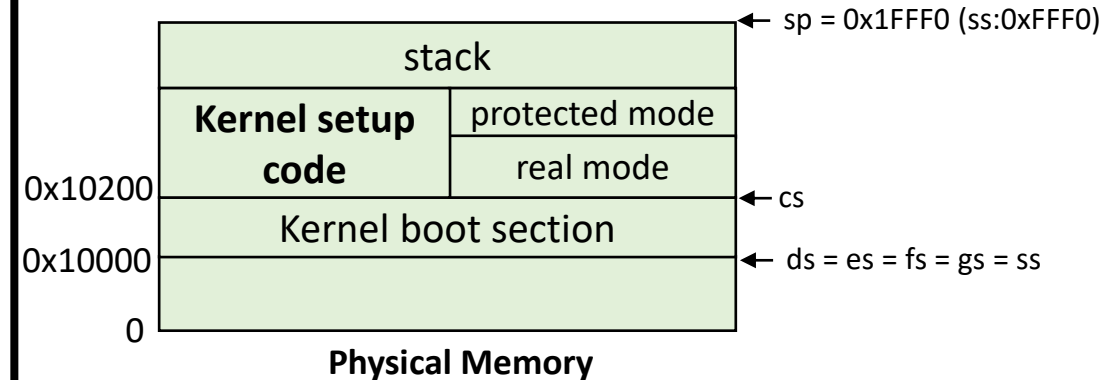
```
IP: 0000 RIP:00000000
CS:IP: 1020:0000 (0x10200)
SS:SP: 1000:FFF0 (0x1FFF0)
SS:BP: 1000:0000 (0x10000)

-----[ CPU Control Register]-----
cr0          0x10          [ ET ]
cr3          0x0          [ PDBR=0 PCID=0 ]
efer         0x0          [ ]

-----[ eflags]-----
OF <0>  DF <0>  IF <0>  TF <0>  SF <0>  ZF <1>  AF <0>  PF <1>  CF <0>
ID <0>  VIP <0>  VIF <0>  AC <0>  VM <0>  RF <0>  NT <0>  IOPL <0>

-----[ CODE ]-----
0x10200:      jmp     0x1026c
0x10202:      rex.W
0x10203:      fs     0x10259
0x10206:      lar     (%rax),%eax
0x10209:      add     %al, (%rax)
0x1020b:      add     %al, (%rax)
0x1020d:      adc     %ah, 0x33(%rax)
0x10210:      mov     $0x81,%al
0x10212:      add     %al, 0x100000(%rax)
0x10218:      add     %dl, (%rax)
0x0000000000000000 in ?? ()
(gdb)
```

QEMU loader loads 'setup.bin' at address 0x10000



gdb command file: Add any gdb commands in this file

```
$tail -n 2 gdb-files/gdb-linux-kernel-real-mode.txt
b *0x10200
c
```

Reference (Real-mode entry point): [Vmlinux: anatomy of bzimage and how x86 64 processor is booted](#)

QEMU/gdb at a glance: Console #1/#2

Console #2: step & continue

```
-----[ CODE ]-----
0x10200:    jmp     0x1026c
0x10202:    rex.W
0x10203:    fs jnb  0x10259
0x10206:    lar     (%rax),%eax
0x10209:    add     %al, (%rax)
0x1020b:    add     %al, (%rax)
0x1020d:    adc     %ah, 0x33(%rax)
0x10210:    mov     $0x81,%al
0x10212:    add     %al, 0x100000(%rax)
0x10218:    add     %dil, (%rax)
0x0000000000000000 in ?? ()
1(gdb) si
```

```
-----[ CODE ]-----
0x1026c:    mov     %ds,%eax
0x1026e:    mov     %eax,%es
0x10270:    cld
0x10271:    mov     %ss,%edx
0x10273:    cmp     %eax,%edx
0x10275:    mov     %esp,%edx
0x10277:    je      0x1028f
0x10279:    mov     $0x6f64a10,%edx
0x1027e:    adc     %eax, (%rdx)
0x10280:    xorb    $0x16,-0x75(%rsp,%rax,1)
0x000000000000006c in ?? ()
2(gdb) c
Continuing.
```

Console #1: QEMU: Guest OS

```
Freeing unused kernel image (text/rodata gap) memory: 2044K
Freeing unused kernel image (rodata/data gap) memory: 560K
Run /init as init process
  with arguments:
    /init
    nokaslr
  with environment:
    HOME=/
    TERM=linux
/bin/sh: can't access tty; job control turned off
/ # input: ImExPS/2 Generic Explorer Mouse as /devices/platform/i8042/serio1/in3
tsc: Refined TSC clocksource calibration: 2100.001 MHz
clocksource: tsc: mask: 0xffffffffffffffff max_cycles: 0x1e453155694, max_idle_s
clocksource: Switched to clocksource tsc

/ #
/ #
/ #
```

Reference (Real-mode entry point): [Vmlinux: anatomy of bzimage and how x86 64 processor is booted](#)

Note: Debug the decompressed vmlinux (generic kernel)

```
$ head -n 12 gdb-files/gdb-linux-kernel-real-mode.txt
# debug info about real-mode code of Linux kernel
add-symbol-file /home/adrian/work/gdb-linux-real-mode/out/obj/linux/arch/x86/boot/setup.elf 0x103ff -s .bstext
0x10000 -s .bsdata 0x1002d -s .header 0x101ef -s .entrytext 0x1026c -s .inittext 0x102d4 -s .initdata 0x103e1 -s .text32
0x130ce -s .bss 0x136e0 -s .data 0x13660

# debug info about compressed vmlinux
add-symbol-file /home/adrian/work/gdb-linux-real-mode/out/obj/linux/arch/x86/boot/compressed/vmlinux 0x3ce4f0 -
s .head.text 0x100000 -s .data 0x3d5b90 -s .bss 0x3d5e40 -s .pgtable 0x3f6000
target remote :1234

# Uncomment the following line if you want to debug the decompressed vmlinux
add-symbol-file /home/adrian/work/gdb-linux-real-mode/out/obj/linux/vmlinux

set print pretty on
```

Kernel supported gdb functions and commands

```
adrian@adrian-ubuntu:gdb-linux-real-mode$ head -n 2 ~/.gdbinit
# Debug kernel
add-auto-load-safe-path /home/adrian/work/gdb-linux-real-mode/out/obj/linux
```

```
(gdb) apropos lx
function lx_clk_core_lookup -- Find struct clk_core by name
function lx_current -- Return current task.
function lx_device_find_by_bus_name -- Find struct device by bus and name (both
strings)
function lx_device_find_by_class_name -- Find struct device by class and name (b
oth strings)
function lx_module -- Find module by name and return the module variable.
function lx_per_cpu -- Return per-cpu variable.
function lx_rb_first -- Lookup and return a node from an RBTREE
function lx_rb_last -- Lookup and return a node from an RBTREE.
function lx_rb_next -- Lookup and return a node from an RBTREE.
function lx_rb_prev -- Lookup and return a node from an RBTREE.
function lx_task_by_pid -- Find Linux task by PID and return the task_struct var
iable.
function lx_thread_info -- Calculate Linux thread_info from task variable.
function lx_thread_info_by_pid -- Calculate Linux thread_info from task variable
found by pid
lx-clk-summary -- Print clk tree summary
lx-cmdline -- Report the Linux Commandline used in the current kernel.
lx-configdump -- Output kernel config to the filename specified as the command
lx-cpus -- List CPU status arrays
lx-device-list-bus -- Print devices on a bus (or all buses if not specified)
lx-device-list-class -- Print devices in a class (or all classes if not specifie
d)
lx-device-list-tree -- Print a device and its children recursively
lx-dmesg -- Print Linux kernel log buffer.
lx-fdttdump -- Output Flattened Device Tree header and dump FDT blob to the filen
ame
lx-genpd-summary -- Print genpd summary
lx-iomem -- Identify the IO memory resource locations defined by the kernel
lx-ioports -- Identify the IO port resource locations defined by the kernel
lx-list-check -- Verify a list consistency
lx-lsmod -- List currently loaded modules.
lx-mounts -- Report the VFS mounts of the current process namespace.
lx-ps -- Dump Linux tasks.
lx-symbols -- (Re-)load symbols of Linux kernel and currently loaded modules.
lx-timerlist -- Print /proc/timer_list
lx-version -- Report the Linux Version of the current kernel.
```


Example #1: Conditional breakpoint

gdb-files/gdb-linux-kernel-real-mode.txt

```
# debug info about real-mode code of Linux kernel
add-symbol-file /home/adrian/work/gdb-linux-real-mode/out/obj/linux/arch/x86/boot/setup.elf 0x103ff -s .btext 0x10000 -s .bss 0x1002d -s .header 0x101ef -s .entrytext 0x1026c -s .inittext 0x102d4 -s .initdata 0x103e1 -s .text32 0x130ce -s .bss 0x136e0 -s .data 0x13660

# debug info about compressed vmlinux
add-symbol-file /home/adrian/work/gdb-linux-real-mode/out/obj/linux/arch/x86/boot/compressed/vmlinux 0x3ce4e0 -s .head.text 0x100000 -s .data 0x3d5b70 -s .bss 0x3d5e00 -s .pgtable 0x3f6000
target remote :1234

# Uncomment the following line if you want to debug the decompressed vmlinux
add-symbol-file /home/adrian/git-repo/gdb-linux-real-mode/out/obj/linux/vmlinux

set print pretty on

# start_of_setup is the entry point in .entrytext section of setup.elf
#b start_of_setup

# startup_32 is the entry point in compressed vmlinux
#b startup_32

#b *0x10200

b do_fault if ($_strexp($_lx_current()->comm, "t_mmap") && vmf->address == 0x7ffff9000)
c
```

Console #1: QEMU: Guest OS

```
// # ./t_mmap ./test.txt
```

Console #2: gdb console

```
Breakpoint 1 at 0xffffffff810dc7d0: file /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/memory.c, line 4113.
[Switching to Thread 1.2]
-----[ STACK ]-----
0x1b1fde8:      Error while running hook_stop:
Cannot access memory at address 0x1b1fde8

Thread 2 hit Breakpoint 1, do_fault (vmf=vmf@entry=0xffffc90001b1fe08) at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/memory.c:4113
4113      struct vm_area_struct *vma = vmf->vma;
(gdb) p /x vmf->address
$1 = 0x7ffff9000
(gdb) p $lx_current()->comm
$2 = "t_mmap\000\0\000\000\000\000\000\000"
(gdb) 
```

Example #2: watchpoint

1

```
(gdb) p *pvmw
$11 = {
  page = 0xffffea00083d5180,
  vma = 0xffff88810089c0c0,
  address = 139817161609216,
  pmd = 0xffff88810036ce10,
  pte = 0xffff8881809c0028,
  ptl = 0x0 <fixed_percpu_data>,
  flags = 0
}
(gdb) p ((struct page *) 0xffffea00083d5180)->_mapcount
$12 = {
  counter = 0
}
(gdb) p /x *(unsigned long *) 0xffff8881809c0028
$13 = 0x800000020f546067
```

Only one pte mapped

pte: physical address

3

```
(gdb) c
Continuing.

-----[ STACK ]-----
0x1ffb30:      Error while running hook_stop:
Cannot access memory at address 0x1ffb30

Thread 4 hit Hardware watchpoint 5: *(0xffff8881809c0028)

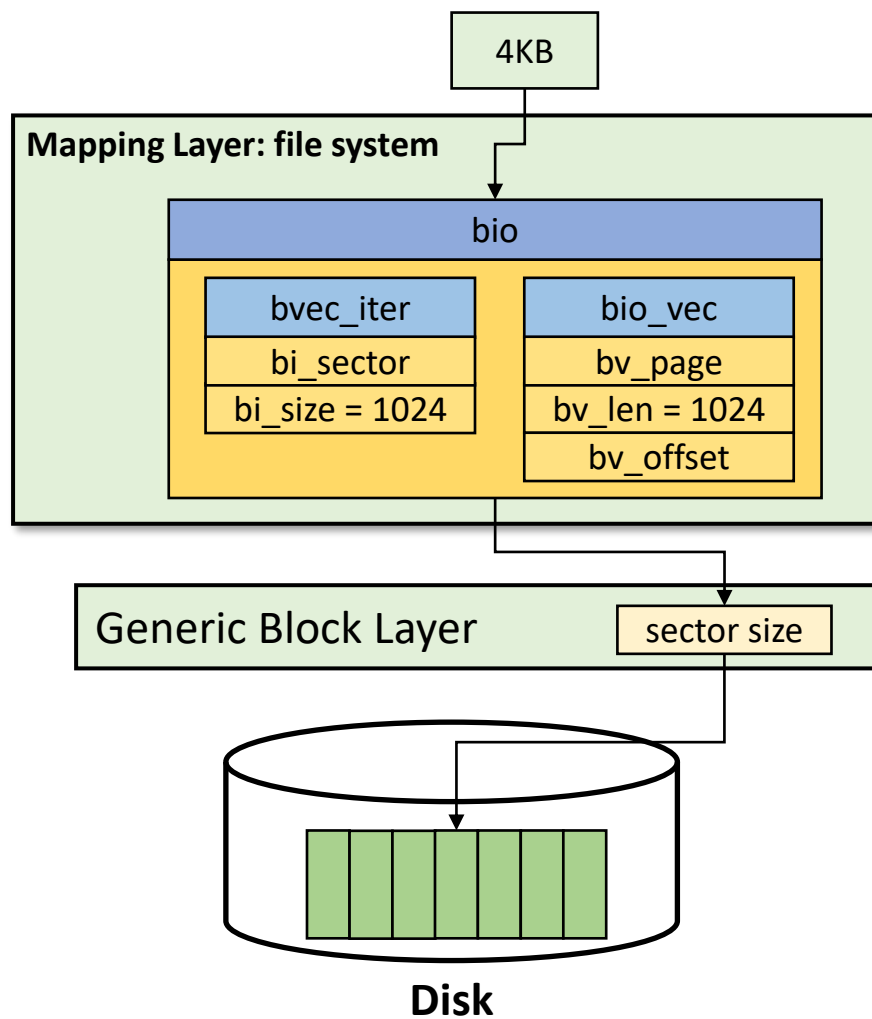
Old value = <unreadable>
New value = -1105890816
try_to_unmap_one (page=0xffffea00083d5180, vma=<optimized out>,
  address=<optimized out>, arg=<optimized out>)
  at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/include
  /asm/pgtable_64.h:67
67      WRITE_ONCE(*ptep, pte);
```

2

```
(gdb) info b
Num      Type      Disp Enb Address      What
1        breakpoint keep y   0xfffffffff81001760 in emulate_vsycall
                                     at /home/adrian/git-repo/gdb-
linux-real-mode/src/linux-5.11/arch/x86/entry/vsyscall/vsyscall_64.c:131
2        breakpoint keep y   0xfffffffff81116959 in __unmap_and_move
                                     at /home/adrian/git-repo/gdb-
linux-real-mode/src/linux-5.11/arch/x86/include/asm/current.h:15
3        breakpoint already hit 1 time
3        breakpoint keep y   0xfffffffff810e9f10 in page_vma_mapped_walk
                                     at /home/adrian/git-repo/gdb-
linux-real-mode/src/linux-5.11/mm/page_vma_mapped.c:145
5        hw watchpoint keep y   *(0xffff8881809c0028)
```

gdb command: (gdb) watch *(0xffff8881809c0028)

Example #3: Data structure examination

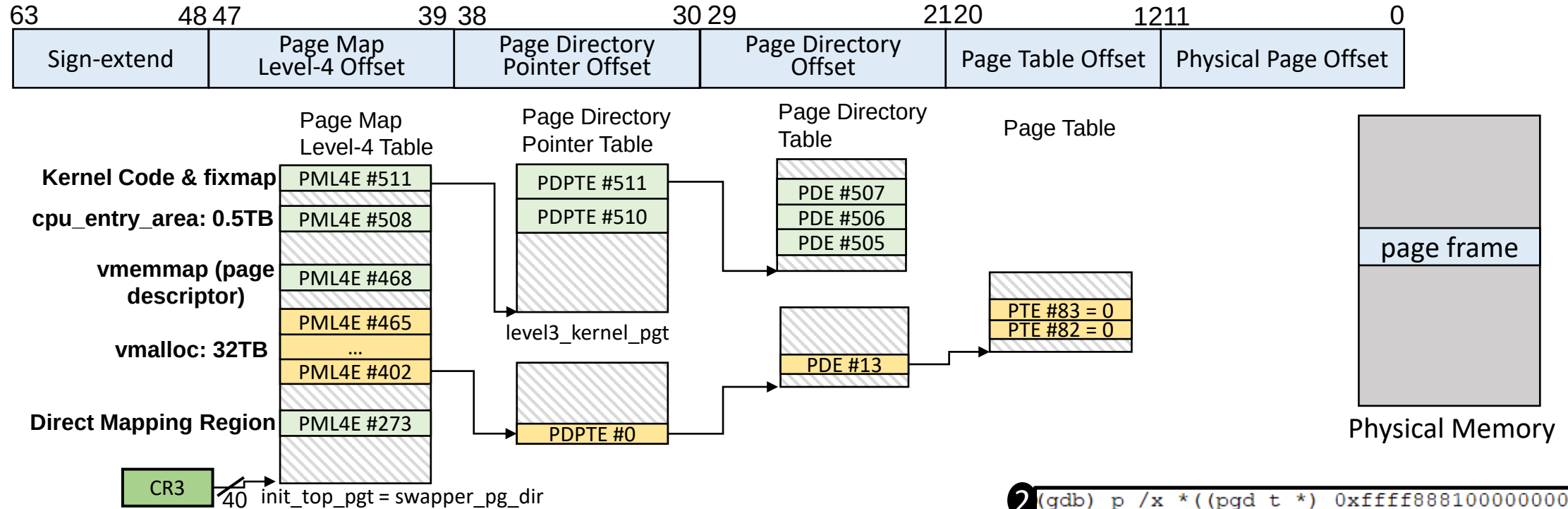


```
(gdb) bt
#0  0xffffffff811a9f8c in __bio_add_page (bio=bio@entry=0xffff8881047136c0, page=
=<optimized out>, len=len@entry=4096, off=off@entry=0) at /home/adrian/git-repo/
gdb-linux-real-mode/src/linux-5.11/arch/x86/include/asm/bitops.h:206
#1  0xffffffff811a9fde in bio_add_page (bio=bio@entry=0xffff8881047136c0, page=p
age@entry=0xffffea0007e04440, len=len@entry=4096, offset=offset@entry=0) at /hom
e/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/block/bio.c:941
#2  0xffffffff811804a6 in ext4_mpage_readpages (inode=<optimized out>, rac=<opti
mized out>, page=<optimized out>, page@entry=0x0 <fixed_percpu_data>) at /home/a
drian/git-repo/gdb-linux-real-mode/src/linux-5.11/fs/ext4/readpage.c:387
#3  0xffffffff81161d5e in ext4_readahead (rac=<optimized out>) at /home/adrian/g
it-repo/gdb-linux-real-mode/src/linux-5.11/fs/ext4/inode.c:3261
```

```
(gdb) p bio->bi_iter
$7 = {
    bi_sector = 12828,
    bi_size = 1024,
    bi_idx = 0,
    bi_bvec_done = 0
}
(gdb) p *bio->bi_io_vec
$8 = {
    bv_page = 0xffffea0007e01180,
    bv_len = 1024,
    bv_offset = 0
}
```

Example #4: Page Table Examination

[Linear Address] 0xffff_c900_01a5_2000, 0xffff_c900_01a5_3000



```
(gdb) bt 1
#0 0xffffffff810f284b in map_kernel_range (pages=<optimized out>, prot=...,
size=<optimized out>, start=<optimized out>)
at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/include/linux/vm
alloc.h:150
(More stack frames follow...)
(gdb) p $lx_current()->mm->pgd
$9 = (pgd_t *) 0xffff8881021e8000
1 (gdb) p /x *((pgd_t *) 0xffff8881021e8000 + 402)
$10 = {
  pgd = 0x100000067
}
```

```
2 (gdb) p /x *((pgd_t *) 0xffff888100000000 + 0)
$13 = {
  pgd = 0x10005a067
}
3 (gdb) p /x *((pgd_t *) 0xffff88810005a000 + 13)
$14 = {
  pgd = 0x1021b7067
}
4 (gdb) p /x *((pgd_t *) 0xffff8881021b7000 + 82)
$15 = {
  pgd = 0x0
}
5 (gdb) p /x *((pgd_t *) 0xffff8881021b7000 + 83)
$16 = {
  pgd = 0x0
}
```

Example #5: ptype: print data type

```
(gdb) ptype /o struct page
/* offset | size */ type = struct page {
/* 0 | 8 */ unsigned long flags;
/* 8 | 40 */ union {
/* 40 */ struct {
/* 8 | 16 */ struct list_head {
/* 8 | 8 */ struct list_head *next;
/* 16 | 8 */ struct list_head *prev;

/* total size (bytes): 16 */
} lru;
/* 24 | 8 */ struct address_space *mapping;
/* 32 | 8 */ unsigned long index;
/* 40 | 8 */ unsigned long private;

/* total size (bytes): 40 */
};
/* 8 | 8 */ struct {
/* 8 | 8 */ dma_addr_t dma_addr;

/* total size (bytes): 8 */
};
...
/* total size (bytes): 64 */
}
```

Example #6: macro

```
(gdb) p /x PAGE_SIZE
$2 = 0x1000
(gdb) macro expand PAGE_SIZE
expands to: ((1UL) << 12)
(gdb) info macro PAGE_SIZE
Defined at /home/adrian/work/gdb-linux-real-mode/src/linux-5.11/arch/x86/include
/asm/page_types.h:11
    included at /home/adrian/work/gdb-linux-real-mode/src/linux-5.11/arch/x86/incl
ude/asm/page.h:9
    included at /home/adrian/work/gdb-linux-real-mode/src/linux-5.11/arch/x86/incl
ude/asm/thread_info.h:12
    included at /home/adrian/work/gdb-linux-real-mode/src/linux-5.11/include/linux
/thread_info.h:58
    included at /home/adrian/work/gdb-linux-real-mode/src/linux-5.11/arch/x86/incl
ude/asm/preempt.h:7
    included at /home/adrian/work/gdb-linux-real-mode/src/linux-5.11/include/linux
/preempt.h:78
    included at /home/adrian/work/gdb-linux-real-mode/src/linux-5.11/include/linux
/smp.h:80
    included at /home/adrian/work/gdb-linux-real-mode/src/linux-5.11/include/linux
/kernel_stat.h:5
    included at /home/adrian/work/gdb-linux-real-mode/src/linux-5.11/mm/memory.c:4
2
#define PAGE_SIZE (_AC(1,UL) << PAGE_SHIFT)
(gdb) □
```